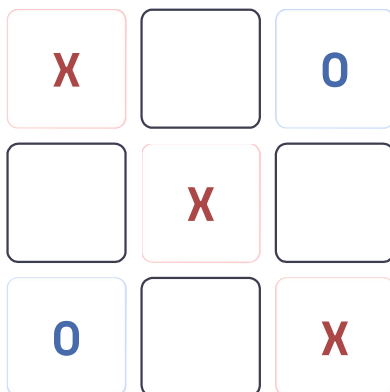


El Gato | Tic Tac Toe - Enunciado

EJERCICIO EVALUADO · FULLSTACK I · JAVA 21 + SPRING BOOT 4



El X/O Distribuido

Dos microservicios. Un árbitro. Dos jugadores. Reglas reales.
El Juego del Gato como sistema distribuido con lógica de negocio,
multi-server, partidas temporizadas y tolerancia a fallos.

Curso Fundamentos Fullstack I

Stack Java 21 · Spring Boot 4.x · MySQL

Modalidad Parejas o individual

REST Richardson Nivel 2

Instancias mínimas 1 server + 2 clientes

[Ver especificación técnica →](#)

Plataforma de juegos en tiempo real – Fase 1: Motor de partidas

GatoLabs está construyendo una plataforma de juegos multijugador en línea. El primer juego en lanzarse es el Gato — simple en sus reglas, pero suficientemente exigente para validar la arquitectura completa de la plataforma antes de escalar a juegos más complejos. Esta decisión no es casual: el Gato tiene estado compartido, turnos alternados, condiciones de término bien definidas y la necesidad de coordinar a dos participantes que actúan desde sistemas distintos. Todos los problemas reales están ahí, en pequeño.

// EL PROBLEMA QUE SE ESTÁ RESOLVIENDO

Cuando dos personas juegan al Gato en una hoja de papel, el árbitro es implícito: ambos ven el mismo tablero, ambos saben de quién es el turno, y cualquiera puede verificar si alguien hizo trampa. En un sistema distribuido, nada de eso es gratis. Cada jugador opera desde su propio dispositivo, con su propia conexión, y no existe un tablero físico compartido.

Alguien tiene que ser el árbitro. Alguien tiene que custodiar el estado oficial de la partida, validar que cada jugada sea legítima, detectar cuándo terminó el juego y comunicarlo a ambos lados. Y ese árbitro tiene que funcionar aunque uno de los jugadores se desconecte, aunque la red falle momentáneamente, aunque dos jugadores intenten mover al mismo tiempo.

"El juego del Gato es trivial. Coordinar dos actores que compiten sobre un estado compartido, en tiempo real, con validaciones de negocio y tolerancia a fallos — eso no lo es. Y es exactamente lo que se hace en sistemas de reservas, subastas, turnos médicos, mercados financieros y cualquier plataforma donde múltiples usuarios actúan sobre el mismo recurso."

La plataforma resuelve esto con dos servicios bien delimitados: un **servidor de juego** que es el árbitro único y autoritativo de todas las partidas, y un **cliente de jugador** que es la interfaz a través de la cual cada persona interactúa con el sistema. Esta separación no es un requisito académico — es el patrón que usan plataformas como Chess.com, Lichess, Steam y cualquier sistema de matchmaking online.

// POR QUÉ DOS MICROSERVICIOS Y NO UNO

La pregunta es legítima: ¿por qué no hacer un solo sistema que gestione partidas y jugadores juntos? La respuesta está en las responsabilidades. El servidor de juego no debería saber nada sobre cómo se autentica un jugador, qué interfaz usa, ni desde qué dispositivo se conecta. Solo le importa: ¿es tu

turno? ¿la casilla está libre? ¿alguien ganó? Eso le permite ser reemplazado, escalado o reescrito sin tocar la lógica de los jugadores.

El cliente, por su parte, es el que conoce al jugador: sus credenciales, su historial local, en qué servers está jugando. Actúa como un agente que representa al jugador ante uno o varios servidores. Un jugador podría tener partidas activas en distintos servidores simultáneamente — el cliente lo gestiona sin que el jugador tenga que pensar en URLs ni en autenticación de servicios.

Esta frontera entre "quién soy" y "qué estoy jugando" es la misma que existe entre un servicio de identidad y un servicio de negocio en cualquier arquitectura moderna.

// EL ENCARGO CONCRETO

Se te encarga implementar la **Fase 1** de la plataforma: el motor de partidas completo. Esto incluye el servidor de juego con todas sus reglas, el cliente como proxy inteligente para el jugador, la comunicación segura entre ambos servicios, y el despliegue containerizado del sistema completo.

La plataforma debe soportar desde el primer día que un jugador pueda jugar en múltiples servidores simultáneamente — porque GatoLabs planea operar servidores regionales. El cliente no puede estar atado a un servidor fijo.

El servidor debe verificar que quien le habla es realmente un cliente autorizado de la plataforma, no cualquier aplicación que adivine el endpoint. El sistema de registro automático de clientes al arranque es el mecanismo que lo garantiza.

La plataforma debe seguir respondiendo con claridad cuando un servidor falla — un jugador no puede recibir un error críptico porque el servidor de turno se cayó. El cliente absorbe ese fallo y lo comunica de forma entendible.

// RESTRICCIONES DEL PROYECTO

Stack no negociable Java 21, Spring Boot 4.x, MySQL, Flyway, Docker. No hay alternativas tecnológicas en esta fase.	Mínimo 3 instancias corriendo 1 servidor de juego y 2 clientes independientes. El flujo completo debe funcionar con los tres activos.
Sin estado compartido entre instancias de cliente Cada cliente tiene su propia base de datos. No se comparte información entre instancias del mismo servicio.	API pública documentada Swagger UI funcional en cada servicio. La API es el contrato — si no está documentada, no existe.
Fallos explícitos, no silenciosos Cualquier fallo de comunicación entre servicios	Deploy obligatorio El sistema debe estar desplegado y accesible

se comunica con un código HTTP correcto y un cuerpo descriptivo. Nunca un stack trace expuesto.

públicamente antes de la entrega. Railway o Render son las plataformas aprobadas.

// LO QUE SE ENTREGA AL FINALIZAR

- ▶ Dos microservicios funcionales, cada uno en su propio módulo del repositorio, con su Dockerfile y migraciones Flyway.

-
- ▶ Un `docker-compose.yml` en la raíz que levanta el sistema completo con `docker compose up` .

-
- ▶ Colección Postman exportada con todos los endpoints, ejemplos de éxito y de error, y un environment para producción.

-
- ▶ Al menos un servidor y un cliente desplegados en Railway o Render con URL pública funcional.

-
- ▶ Tests unitarios que pasan `mvn test` — validaciones del juego, detector de victoria, fallback del cliente.
-

